

Data 607 Week 11

Benjamin Bravo

2026-04-20

Approach

For this assignment, I will build a personalized recommender system using the movie ratings dataset provided. Instead of giving the same recommendations to every user, the goal is to create personalized suggestions based on user preferences. I plan to use User-Based Collaborative Filtering, which compares users with similar rating patterns and recommends items that similar users enjoyed.

To complete the assignment, I will first load and clean the dataset in R. Then I will convert the ratings into the correct matrix format needed for the recommender model. After that, I will split the data into training and testing sets so the model can be evaluated properly. Once the model is built, I will generate top recommendations for users and measure prediction accuracy using metrics such as RMSE.

One data challenge I foresee is the small size of the dataset. With fewer users and ratings, some users may not have enough information for the model to compare similarities accurately. Another challenge is missing values, since many users may not rate every movie. This can make recommendations harder to generate. To address these issues, I will use cross-validation and lower the number of required known ratings during evaluation. I will also allow the model to automatically select users who still have unrated items available for recommendations.

Overall, my approach will focus on cleaning the data, building a collaborative filtering model, evaluating its performance, and generating personalized recommendations in a simple and effective way.

Introduction

For this assignment, I used MovieRatings as the dataset. I built a personalized recommender system using User-to-User Collaborative Filtering. This method compares users who rate items similarly and recommends items based on shared preferences.

The recommender system outputs the Top recommended items for each user. To evaluate the model, I used predicted ratings and measured accuracy with RMSE (Root Mean Squared Error). Lower RMSE values mean better predictions.

```
install.packages("readxl")
```

```
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'  
## (as 'lib' is unspecified)
```

```
install.packages("recommenderlab")
```

```
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'  
## (as 'lib' is unspecified)
```

```
install.packages("caret")
```

```
## Installing package into '/cloud/lib/x86_64-pc-linux-gnu-library/4.5'  
## (as 'lib' is unspecified)
```

```
library(readxl)
library(recommenderlab)
```

```
## Loading required package: Matrix
## Loading required package: arules
##
## Attaching package: 'arules'
## The following objects are masked from 'package:base':
##
##      abbreviate, write
## Loading required package: proxy
##
## Attaching package: 'proxy'
## The following object is masked from 'package:Matrix':
##
##      as.matrix
## The following objects are masked from 'package:stats':
##
##      as.dist, dist
## The following object is masked from 'package:base':
##
##      as.matrix
```

```
library(caret)
```

```
## Loading required package: ggplot2
## Loading required package: lattice
##
## Attaching package: 'caret'
## The following objects are masked from 'package:recommenderlab':
##
##      MAE, RMSE
```

```
url <- "https://raw.githubusercontent.com/bb2955/Data-607/main/MovieRatings%20(1).xlsx"
download.file(url, destfile = "MovieRatings.xlsx", mode = "wb")
```

```
ratings_data <- read_excel("MovieRatings.xlsx")
```

```
ratings_data
```

```
## # A tibble: 16 x 7
##   Critic    CaptainAmerica Deadpool Frozen JungleBook PitchPerfect2 StarWarsForce
##   <chr>          <dbl>    <dbl>  <dbl>    <dbl>        <dbl>        <dbl>
## 1 Burton            NA      NA    NA         4            NA            4
## 2 Charley           4        5     4         3            2            3
## 3 Dan              NA        5    NA         NA           NA            5
## 4 Dieudo~          5        4    NA         NA           NA            5
## 5 Matt             4       NA     2         NA            2            5
## 6 Mauric~          4       NA     3         3            4           NA
## 7 Max              4        4     4         2            2            4
```

## 8 Nathan	NA	NA	NA	NA	NA	4
## 9 Param	4	4	1	NA	NA	5
## 10 Parshu	4	3	5	5	2	3
## 11 Prasha~	5	5	5	5	NA	4
## 12 Shipra	NA	NA	4	5	NA	3
## 13 Sreeja~	5	5	5	4	4	5
## 14 Steve	4	NA	NA	NA	NA	4
## 15 Vuthy	4	5	3	3	3	NA
## 16 Xingjia	NA	NA	5	5	NA	NA

```
ratings_df <- as.data.frame(ratings_data)
row.names(ratings_df) <- ratings_df[,1]
ratings_df <- ratings_df[,-1]
```

```
ratings_df[] <- lapply(ratings_df, as.numeric)
```

```
ratings_matrix <- as.matrix(ratings_df)
```

```
ratings <- as(ratings_matrix, "realRatingMatrix")
```

```
scheme <- evaluationScheme(ratings,
                           method = "cross-validation",
                           k = 3,
                           given = 1)
```

```
model <- Recommender(getData(scheme, "train"),
                    method = "UBCF")
```

```
pred <- predict(model,
               getData(scheme, "known"),
               type = "ratings")
```

```
accuracy <- calcPredictionAccuracy(pred,
                                   getData(scheme, "unknown"))
```

```
accuracy
```

```
##      RMSE      MSE      MAE
## 1.905683 3.631628 1.572527
```

```
final_model <- Recommender(ratings, method = "UBCF")
```

```
for(i in 1:nrow(ratings)){

  top3 <- predict(final_model, ratings[i], n = 3)
  rec_list <- as(top3, "list")

  if(length(rec_list[[1]]) > 0){

    cat("Top 3 Recommendations for", rownames(ratings_matrix)[i], ":\n")
    print(unlist(rec_list))
    break

  }

}
```

```
## Top 3 Recommendations for Dieudonne :
##           01           02           03
##   "JungleBook"   "Frozen" "PitchPerfect2"
```

Code Explanation

First, the required libraries were loaded. The `readxl` package was used to read the Excel file directly from GitHub, and the `recommenderlab` package was used to build the recommendation system. The ratings file was downloaded and imported into R. The first column contained the user names, so it was converted into row names, while the remaining columns contained the movie ratings.

Next, all ratings were converted into numeric values and changed into a matrix format. This matrix was then converted into a `realRatingMatrix`, which is the format needed by the recommender system package. In this structure, each row represents a user and each column represents a movie. Missing values represent movies the user has not rated.

After preparing the data, I created an evaluation split using cross-validation. This divided the dataset into training and testing portions so the model could be tested on unseen data. I used `given = 1`, which means one rating was kept visible for each user while the remaining ratings were hidden for testing. This was necessary because the dataset is small and some users had limited ratings.

Then, I built a User-Based Collaborative Filtering (UBCF) model. This method compares users who have similar rating patterns. If similar users liked movies that another user has not rated yet, those movies are recommended to that user. Predictions were made on the hidden test data, and the model's performance was measured using RMSE, MSE, and MAE.

Finally, a new recommender model was trained using the full dataset so final recommendations could be created. The code automatically searched for the first user who still had unrated movies available. For that user, the system generated the top three personalized recommendations: JungleBook, Frozen, and PitchPerfect2.

Conclusion

This assignment successfully created a personalized recommender system using User-Based Collaborative Filtering. Unlike a global baseline model that gives the same recommendations to everyone, this model used user similarities to make recommendations tailored to each person. The model was evaluated using RMSE, MSE, and MAE, showing that it was able to make reasonable predictions on hidden ratings.

The final recommendation output showed that the user Dieudonne was recommended JungleBook, Frozen, and PitchPerfect2 based on rating behavior from similar users. This demonstrates how collaborative filtering can personalize recommendations and improve user experience. Systems like this are commonly used by Netflix, Spotify, and many other platforms.